

Front end

Integração com Java Aplicação web

Se a aplicação tem **backend em Java (Spring Boot)**, a melhor escolha de front-end depende do tipo de sistema que está em desenvolvendo e da experiência da equipe. Hoje, a recomendação seria a seguinte:

Front-end	Nível	Integração com Java	Curva de aprendizado	Mercado
React	★★★★★	Excelente	Média	Muito alta
Angular	★★★★★	Excelente	Alta	Muito alta (empresas)
Vue.js	★★★★☆	Excelente	Baixa	Alta
Thymeleaf	★★★☆☆	Nativa	Muito baixa	Média
Vaadin	★★★★☆	Excelente	Muito baixa para quem programa Java	Média

1. React (Minha primeira recomendação)

Para aplicações modernas, SPA (**Single Page Application**) e **APIs REST** com Spring Boot, React é uma excelente opção.

Arquitetura



Vantagens:

- enorme comunidade;
- milhares de componentes prontos;
- fácil integração com Spring Security + JWT;
- excelente para dashboards;
- muito requisitado no mercado.

Bibliotecas comuns:

- React
- Vite
- React Router
- Axios
- Material UI
- Tailwind CSS

2. Angular (Muito usado em empresas)

Angular também é uma ótima escolha para ambiente corporativo e acadêmico.

Foi desenvolvido justamente para aplicações grandes.

Possui:

- TypeScript obrigatório
- organização excelente
- injeção de dependência
- componentes
- módulos
- testes

Integra perfeitamente com:

- Spring Boot
- Spring Security
- OAuth2
- JWT

Nota: Muitas instituições financeiras e grandes empresas utilizam Angular.

3. Vue.js

É o mais simples dos três. Muito produtivo.

Nota: Alunos iniciantes, melhor escolha **Vue** antes de **React**.

4. Thymeleaf

Aplicação tradicional (**renderização no servidor**), **Thymeleaf** excelente.

Exemplo:

Spring Boot

@Controller



Thymeleaf



HTML

Nota: Não precisa criar **API REST**.

Ideal para:

- sistemas internos
- CRUD
- ERP
- aplicações administrativas

5. Vaadin

Vale conhecer para desenvolver o **front end** em Java.

Exemplo:

Button btn = new **Button**("Salvar");

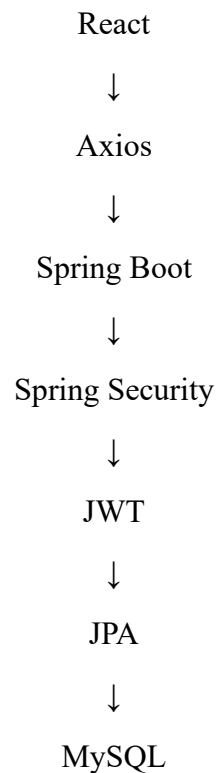
Nota: Sem escrever HTML ou JavaScript na maior parte do tempo; Muito utilizado em sistemas corporativos.

Opção de Mercado de Trabalho

Tools:

- Spring Boot
- Java 21
- Maven
- APIs REST
- H2
- MySQL
- Feign
- autenticação
- sistemas corporativos

Nesse cenário:



Nota: Arquitetura moderna, escalável e amplamente utilizada.

Ferramentas que combinam muito bem

- **IDE Java:** STS 4 ou IntelliJ IDEA
- **Front-end:** Visual Studio Code
- **Back-end:** Spring Boot
- **Banco:** MySQL ou PostgreSQL
- **Testes de API:** Postman ou Insomnia
- **Controle de versão:** Git + GitHub

Para projetos novos atualmente:

- **Back-end:** Java 21
- **Spring Boot 3.5+**
- Spring Security
- JWT
- Spring Data JPA
- MySQL

Front-end:

- React 19
- Vite
- TypeScript
- Axios
- React Router
- Material UI (ou Tailwind CSS, se preferir um visual mais customizado)

Essa combinação oferece ótimo desempenho, boa produtividade e excelente alinhamento com o mercado.

React traz um benefício adicional: além de ser muito usado profissionalmente, permite demonstrar conceitos importantes de desenvolvimento moderno, como componentes, gerenciamento de estado, consumo de **APIs REST** e autenticação baseada em **JWT**, que se integram de forma bastante natural ao ecossistema **Spring Boot**.

Único repositório GitHub? back e front? monolítico?

Duas estratégias bastante utilizadas.

Monorepositório (um único repositório GitHub) contendo o back-end e o front-end separados em pastas.

Por exemplo:

securitycontabil/

```

|
|— backend/
|   |— src/
|   |— pom.xml
|   └─ ...
|
|— frontend/
|   |— src/
|   |— package.json
|   └─ ...
|
|— docs/
|
|— docker-compose.yml
|
|— README.md
|
└─ .gitignore
  
```

Vantagens:

- um único repositório para clonar;
- controle de versões sincronizado entre front e back;
- mais simples para um único desenvolvedor ou uma equipe pequena;
- facilita CI/CD e documentação.

Nota: **Monolítico não significa front e back juntos**

Há uma confusão comum entre **monólito** e **monorepositório**.

Monólito

É uma única aplicação.

Exemplo:

Spring Boot
 +
Thymeleaf
 =
 1 aplicação
 1 deploy
 1 artefato (**JAR/WAR**)

Nota: Tudo é executado dentro do **Spring Boot**.

SPA + API

São **duas aplicações**:

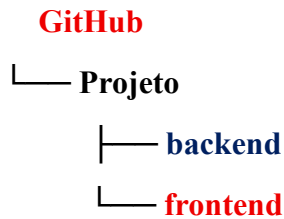
React
 ↓
REST API
 ↓
 Spring Boot

Nota: Mesmo que estejam no mesmo **GitHub**, continuam sendo **duas aplicações independentes**.

Cada uma possui:

- seu processo;
- suas dependências;
- seu build;
- seu deploy.

Portanto:



Nota: Não é um monólito; é um **monorepositório** com arquitetura cliente-servidor.

Separar em dois repositórios

Nota: Grandes empresas costumam adotar:

Gthub

empresa-api

empresa-front

Isso faz sentido quando:

- equipes diferentes cuidam do **front** e do **back**;
- ciclos de implantação são independentes;
- o front atende múltiplas APIs ou vice-versa.

Exemplo Prático de Projeto

Tais como, **SecurityContabil** e outros sistemas administrativos em **Spring Boot**, adotar um único repositório:

SistemaXYZ/

backend/

frontend/

database/

docs/

docker/

README.md

Nota: No **README.md**, documentar como iniciar o ambiente:

git clone ...

npm run dev

cd backend

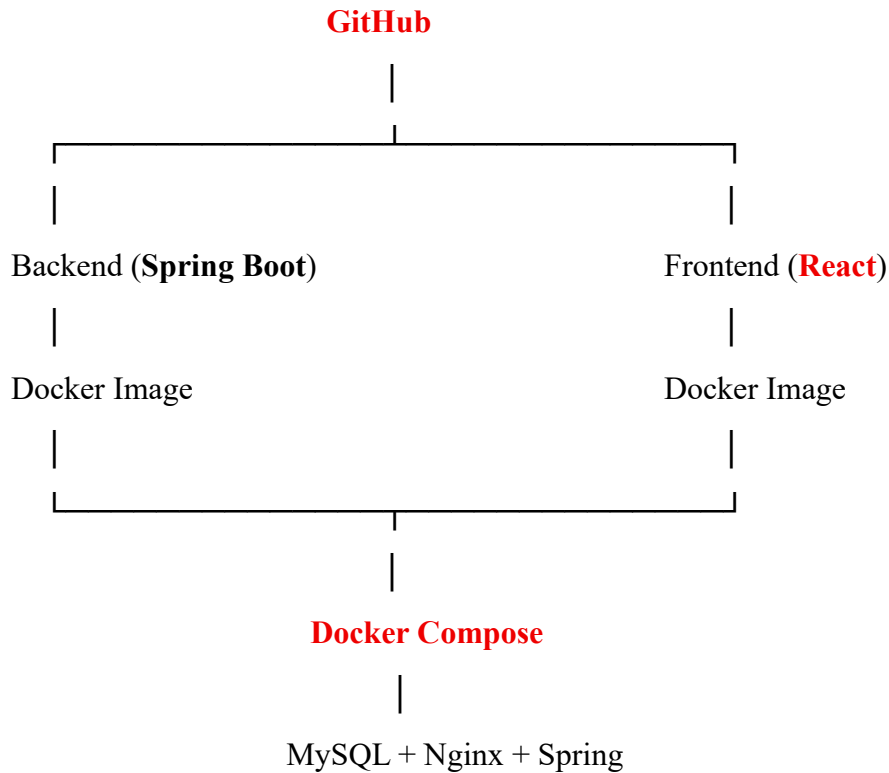
cd ../frontend

mvn spring-boot:run

npm install

Exemplo em produção

Uma arquitetura bastante comum:



Nota: Assim, o código permanece organizado em um **único repositório**, mas **cada aplicação** é construída e **implantada de forma independente**.

Essa abordagem costuma oferecer o melhor equilíbrio entre organização, simplicidade e facilidade de manutenção.

Se o projeto crescer significativamente, sempre é possível separar o **front-end** e o **back-end** em repositórios distintos mais adiante, sem grandes impactos na arquitetura.